

Notification Framework

A Scalable, Multi-Channel Notification Delivery Platform

Group 18

Department of Computer Science & Engineering,
Indian Institute of Technology Hyderabad

May 5, 2026

Channels: Email • WhatsApp • SMS • Push — **Stack:** FastAPI · Celery · RabbitMQ · MongoDB

Outline

- 1 Problem Statement & Motivation
- 2 System Architecture
- 3 Core Features
- 4 API Design
- 5 Rate Limiting & Reliability
- 6 Data Models & Storage
- 7 User Preferences & DND
- 8 Observability & Admin Dashboard
- 9 Technology Stack
- 10 Key Design Decisions
- 11 Conclusion & Future Work

Problem Statement & Motivation

The Challenge

- Modern apps must reach users across **multiple channels**: email, SMS, WhatsApp, push
- Each channel has distinct **providers**, **rate limits**, and **delivery semantics**
- Naive integrations lead to tight coupling and brittle failures
- No unified view of delivery status or user preferences

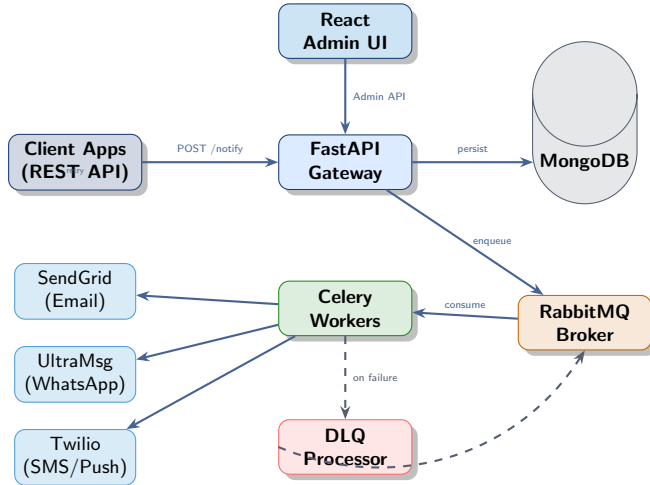
Pain Points Without a Framework

- × Re-implementing provider SDKs per app
- × No retry / Dead-Letter Queue
- × Ignoring user opt-out / DND windows
- × No observability or analytics
- × Missed or duplicate notifications under load

Our Goal

Build a **single reusable backend service** callable via one REST endpoint to deliver notifications reliably at scale.

High-Level System Architecture



Core Features — Delivery, Preferences & Rate Limiting

1. Multi-Channel Delivery

- **Email** via SendGrid (HTML + plain text)
- **WhatsApp** via UltraMsg API
- **SMS & Push** via Twilio
- Configurable provider priority & fallback

2. User Preference & DND

- Per-user channel opt-in/opt-out
- Per-event-type channel mapping
- Do-Not-Disturb time windows (IST-aware)
- Notifications deferred, not dropped

3. Rate Limiting (3 Layers)

- **Global IP-level** sliding window (100 req/min)
- **System-wide** capacity cap (5000 req/min)
- **Per-user / per-client** channel quotas
- Full audit trail persisted to MongoDB

Design Principle

Every layer returns HTTP 429 with a `Retry-After` header so callers can back-off gracefully without polling.

Core Features — Queues, Reliability & Observability

4. Async Queue Architecture

- RabbitMQ as message broker
- Dedicated Celery worker per channel queue
- Admin pause / resume / purge queue controls
- Per-channel configurable throughput rate

5. Reliability & DLQ

- Automatic retry on provider failure
- Dead-Letter Queue for persistent failures
- Admin-triggered DLQ reprocess / discard
- Full delivery log recorded per job attempt

6. Observability

- Real-time delivery stats & latency (p50/p95/p99)
- Webhook callbacks to client apps on delivery
- Rate-limit audit logs stored in MongoDB
- React admin dashboard for operational control

Admin Dashboard

Live queue depth, job status, DLQ entries, provider config, rate-limit logs, and webhook call history — all in one React UI.

Core & Client Management

Method	Route
POST	/notify
GET	/jobs /logs /stats /latency
POST	/clients/register
GET	/clients
PATCH	/clients/{id}
POST	/clients/{id}/rotate-key
PUT	/clients/{id}/webhook-url
POST	/preferences
GET	/preferences/{user.id}

Admin Queue Controls

Method	Route
GET	/queue-stats
POST	/pause-queue
POST	/resume-queue
POST	/update-rate-limit
POST	/clear-queue
POST	/retry-failed
POST	/reprocess-dlq
GET	/dlq /rate-limit-logs
GET	/providers
POST	/providers
GET	/webhooks/ultramsg/latest

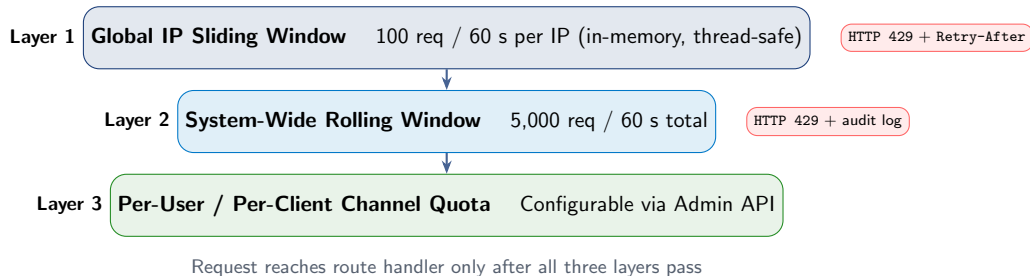
Security

X-API-Key header (HMAC-SHA256 hashed) for notify routes. Bearer token for admin routes. Key rotation supported via /rotate-key.

Webhook Endpoints

POST /webhooks/sendgrid and /webhooks/ultramsg receive provider delivery receipts and update job status in MongoDB.

Three-Layer Rate Limiting Architecture



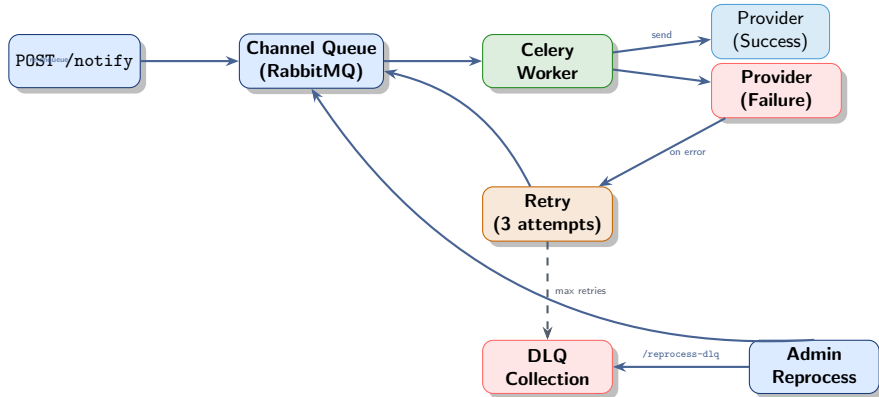
Response Headers (on pass)

X-RateLimit-Remaining-IP: N
X-RateLimit-Remaining-System: N

Persistence

All rate-limit hits written to MongoDB (`rate_limit_logs` & `global_rate_limit_logs`) for audit and analytics.

Reliability: DLQ & Retry Flow



Delivery Log

Every attempt recorded: status, provider, timestamp, error message

DLQ Actions

Admin can **reprocess** (re-enqueue) or **discard** individual DLQ entries

Webhook Callbacks

Client apps notified on delivery success or final failure via `webhook_url`

MongoDB Collections & Data Model

Primary Collections

Collection	Purpose
notification_requests	Incoming request log
notification_jobs	Per-recipient job & status
delivery_logs	Per-attempt audit trail
user_preferences	Channel opt-in + DND
clients	API key store (hashed)
providers	Provider config & priority

Operational Collections

Collection	Purpose
dlq	Dead-letter entries
rate_limit_logs	Per-user RL hits
global_rate_limit_logs	IP / system RL hits
queue_controls	Pause/rate state
webhook_calls	Outbound webhook log

Document Lifecycle

- 1 POST /notify → **notification_request** doc (unique request_id)
- 2 One **notification_job** per recipient × channel
- 3 Celery worker picks job, attempts delivery
- 4 Attempt appended to **delivery_log** (status, latency, error)
- 5 Status: PENDING → IN_PROGRESS → DELIVERED / FAILED / DLQ
- 6 Webhook fired on terminal state

Key Design Choice

Separating *requests* from *jobs* allows one request to fan out to many recipients and channels — each tracked independently.

User Preferences & Do-Not-Disturb

Preference Model

- Global channel enable/disable per user
- **Per-event-type** channel mapping (e.g. `order_update` → `{email, whatsapp}`)
- DND windows: list of (start, end) times in IST
- Stored in MongoDB; updated via `POST /preferences`

Channel Resolution Logic

- 1 Check user opt-in per requested channel
- 2 Apply per-event-type channel override
- 3 Evaluate DND window — defer or suppress if active
- 4 Return resolved list of eligible channels

DND Window Detection

- All times in **IST** (Asia/Kolkata)
- Handles overnight windows (e.g. 22:00–08:00)
- `is_in_dnd()` evaluated at enqueue time
- Future: schedule delivery after DND lifts

Unsubscribe Support

`GET /unsubscribe?user_id=&channel=` — one-click opt-out link embeddable in email footers and WhatsApp messages.

Privacy

Preferences are scoped per `client_id` — same user can have different settings across applications.

Observability: Stats & Latency Analytics

GET /stats **Response**

- Total jobs by status: DELIVERED, FAILED, PENDING, DLQ
- Breakdown by channel and by client
- Provider-level success / failure counts
- Rate-limit hit counts

GET /latency **Response**

- Percentiles: **p50, p95, p99** per channel
- Nearest-rank algorithm over sorted delivery times
- Delta: `enqueued_at` → `delivered_at` from delivery logs

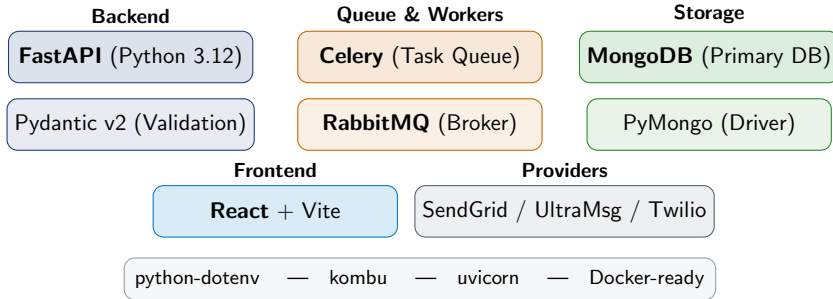
Admin Dashboard (React)

- Live job queue view with status filters
- Queue pause / resume / purge controls
- Per-channel rate limit adjustment
- DLQ management: reprocess or discard
- Provider configuration management
- Rate-limit log viewer
- Webhook call history

Webhook Audit

Every outbound webhook call is logged: URL, HTTP status, response body (capped at 2000 chars), and timestamp — full delivery accountability.

Technology Stack



Key Design Decisions & Trade-offs

Async Delivery — Celery + RabbitMQ

Decouples request acceptance from sending. API returns immediately; workers process in background. Workers scale independently of the API.

Provider Abstraction Layer

Provider configs live in MongoDB. Adding a new provider requires only a DB insert — no code changes. Workers resolve provider priority dynamically.

In-Memory Rate Limiting (DB-backed audit)

Global counters in memory for sub-millisecond checks. Rate-limit hits persisted to MongoDB via a daemon thread — never blocks the response path.

DLQ as First-Class Entity

Failed jobs are moved to a DLQ collection instead of being silently dropped. Admins have full visibility: reprocess or discard, with audit history.

HMAC-SHA256 API Key Hashing

Raw keys returned once at registration, never stored. Rotation supported. Constant-time `hmac.compare_digest` prevents timing attacks.

Known Limitation

Global rate-limit counters reset on process restart. For multi-process / multi-node deployments, migrate to a shared **Redis** store.

What We Built

- Production-grade notification service as a reusable backend
- Three-layer rate limiting with full audit trail
- Async multi-channel delivery (Email, WhatsApp, SMS, Push)
- Dead-letter queue with admin reprocessing
- Per-user preferences & DND window enforcement
- React admin dashboard for operational control
- Webhook callbacks for client-side event tracking
- Latency percentile analytics (p50 / p95 / p99)

Future Enhancements

- **Redis-backed rate counters** for multi-node deployments
- **Scheduled / deferred delivery** (send after DND lifts)
- **JWT-based user authentication** (replace header user_id)
- **Template engine** for standardised message formats
- **Priority queues** (critical alerts bypass normal queue)
- **A/B provider testing** with automatic failover
- **OpenTelemetry** for distributed tracing

Takeaway

A well-designed notification layer significantly reduces integration complexity while improving reliability and user experience across all delivery channels.

Thank You

Notification Framework

FastAPI • Celery • RabbitMQ • MongoDB • React

Channels: Email • WhatsApp • SMS • Push

Scalable • Reliable • Observable

Questions & Discussion

Group 18 — Department of Computer Science & Engineering, IITH
May 5, 2026